1BM24CS104_PSO.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[3] ✓ 0s

```
import random

# ========================================
# STEP 1: System Configuration (Hardcoded)
# ========================================
TARGET_LOAD = 150.0      # Target power demand in kW
C_SOLAR = 0.04           # Solar cost ($/kWh)
C_WIND = 0.06            # Wind cost ($/kWh)
C_BATTERY = 0.09         # Battery cost ($/kWh)
PENALTY_FACTOR = 10.0    # Penalty multiplier for mismatch

# Bounds for [Solar, Wind, Battery]
bounds = [
    (0.0, TARGET_LOAD),              # Solar max cap
    (0.0, TARGET_LOAD),              # Wind max cap
    (-TARGET_LOAD/2, TARGET_LOAD/2)  # Battery range (Charge/Discharge)
]

# Objective Function to Minimize
def objective_function(particle_position):
    P_solar, P_wind, P_battery = particle_position

    # Financial costs
    cost_solar = P_solar * C_SOLAR
    cost_wind = P_wind * C_WIND
    cost_battery = abs(P_battery) * C_BATTERY

    # Load balancing penalty
    total_served = P_solar + P_wind + P_battery
    mismatch = abs(TARGET_LOAD - total_served)

    return (cost_solar + cost_wind + cost_battery) + (mismatch * PENALTY_FACTOR)

# ========================================
# STEP 2: PSO Parameters
# ========================================
N = 40          # Swarm size
w = 0.7         # Inertia weight
c1 = 1.5        # Cognitive weight
```

1BM24CS104_PSO.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[3] ✓ 0s

```
# ========================================
N = 40          # Swarm size
w = 0.7         # Inertia weight
c1 = 1.5        # Cognitive weight
c2 = 1.5        # Social weight
T = 150         # Iterations
dim = 3         # 3 Assets

# ========================================
# STEP 3: Swarm Initialization
# ========================================
particles_pos = []
particles_vel = []
pbest_pos = []
pbest_fitness = []

for _ in range(N):
    pos = [random.uniform(bounds[d][0], bounds[d][1]) for d in range(dim)]
    vel = [random.uniform(-2, 2) for d in range(dim)]

    particles_pos.append(pos)
    particles_vel.append(vel)
    pbest_pos.append(list(pos))
    pbest_fitness.append(float('inf'))

gbest_pos = None
gbest_fitness = float('inf')

# ========================================
# STEP 4-6: Optimization Loop
# ========================================
for t in range(T):
    for i in range(N):
        fitness = objective_function(particles_pos[i])

        if fitness < pbest_fitness[i]:
            pbest_fitness[i] = fitness
            pbest_pos[i] = list(particles_pos[i])

        if fitness < gbest_fitness:
            gbest_fitness = fitness
```

Variables Terminal

2 26°C Mostly cloudy

Search

[3] ✓ 0s

```
            pbest_pos[i] = list(particles_pos[i])

        if fitness < gbest_fitness:
            gbest_fitness = fitness
            gbest_pos = list(particles_pos[i])

    for i in range(N):
        for d in range(dim):
            r1, r2 = random.random(), random.random()

            # Velocity & position updates
            particles_vel[i][d] = (w * particles_vel[i][d] +
                                   c1 * r1 * (pbest_pos[i][d] - particles_pos[i][d]) +
                                   c2 * r2 * (gbest_pos[d] - particles_pos[i][d]))
            particles_pos[i][d] += particles_vel[i][d]

            # Boundary enforcement
            if particles_pos[i][d] < bounds[d][0]:
                particles_pos[i][d] = bounds[d][0]
            if particles_pos[i][d] > bounds[d][1]:
                particles_pos[i][d] = bounds[d][1]

# ========================================
# STEP 7: Final Calculations & Output
# ========================================
opt_solar, opt_wind, opt_battery = gbest_pos

# Calculate actual operational cost (without penalty markup)
total_energy_cost = (opt_solar * C_SOLAR) + (opt_wind * C_WIND) + (abs(opt_battery) * C_BATTERY)

# Calculate exact generation and load metrics
total_renewable_generated = opt_solar + opt_wind
total_system_output = total_renewable_generated + opt_battery

# Utilization % = (Renewable Power Used / Total System Output) * 100
# (If battery is charging, it consumes excess renewables to preserve it)
if opt_battery < 0:
    renewable_utilized = total_renewable_generated
else:
    renewable_utilized = opt_solar + opt_wind
```

1BM24CS104_PSO.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[3] ✓ 0s

```
    renewable_utilized = total_renewable_generated
else:
    renewable_utilized = opt_solar + opt_wind

utilization_percentage = (renewable_utilized / TARGET_LOAD) * 100
if utilization_percentage > 100.0:
    utilization_percentage = 100.0

print("==================================================")
print("          OPTIMIZATION RUN RESULTS                ")
print("==================================================")
print(f"TOTAL ENERGY COST          : ${total_energy_cost:.2f}")
print(f"RENEWABLE ENERGY UTILIZATION: {utilization_percentage:.2f}%")
print("--------------------------------------------------")
print(f"Optimal Asset Dispatch Mix:")
print(f"  -> Solar Power Output    : {opt_solar:.2f} kW")
print(f"  -> Wind Power Output     : {opt_wind:.2f} kW")
print(f"  -> Battery Grid Action   : {opt_battery:.2f} kW")
print("==================================================")
```

```
==================================================
          OPTIMIZATION RUN RESULTS
==================================================
TOTAL ENERGY COST          : $8.43
RENEWABLE ENERGY UTILIZATION: 99.16%
--------------------------------------------------
Optimal Asset Dispatch Mix:
  -> Solar Power Output    : 30.72 kW
  -> Wind Power Output     : 118.02 kW
  -> Battery Grid Action   : 1.30 kW
==================================================
```

Page No.
Date:

Particle - swarm optimization:

Renewable energy optimization:

$x_i = [P_{solar,i}, P_{wind,i}, P_{battery,i}]$

$\min f(x_i) = \text{cost}_{\text{operating}} + \text{Penalty}_{\text{mismatch}}$

Pseudo code:

Initialize N particles with random values for [Solar, Wind, Battery]
Set pbest = current position
Set gbest = position with the lowest initial cost
Repeat for Max_Iterations:
  For each particle:
    Total_Power = Solar + Wind + Battery
    Mismatch = Absolute (Target_Load - Total_Power)
    Cost = (Solar * C_S) + (Wind * C_W) + (|Battery| * C_b)
      + (Mismatch * Penalty)
    If Cost < Personal_Best_Cost:
      pbest = current position
    If Cost < Global_Best_Cost
      gbest = current position
  For each particle:
    Update Velocity using (W, C1, C2, pbest, gbest)
    Update Position = Position + Velocity
    Clamp position values to stay within safety limits
Output gbest (optimal Solar, Wind, and Battery outputs)

16/10/25

Page No.
Date:

o/p:

-- OPTIMIZATION RUN RESULTS --

Total energy cost : $8.43

Renewable Energy utilization : 99.16%.

Optimal Asset Dispatch Mix:

→ Solar Power output : 30.72 kW

→ Wind Power Output : 118.02 kW

→ Battery Grid Action : 1.30 kW

o/p:

15/10/25